DocuGraph AI

Product Requirements Document (PRD)

Field	Value
Version	1.1 (updated from Master Build Spec v1.0)
Date	October 18, 2026
Status	Phase 1 of 7 complete. Phase 2 (Ingestion Pipeline) is next.
Stage	MVP / prototype. Free-tier only, zero running cost.
Audience	Founder, future developers, AI coding assistants (VS Code handoff)

1. Vision

Build a document intelligence system that prioritizes **auditability and transparency** over raw speed. Every answer must show why the AI said it, and let the user verify it against the source document: source chunk, page number, graph path, and a confidence score.

2. Problem Statement

Traditional vector-only RAG fails document-heavy teams in four ways:

- **Context loss:** chunks are indexed in isolation, so relationships between entities on different pages are missed.
- **Hallucination without proof:** the LLM gives plausible answers the user cannot verify.
- **Single retrieval signal:** vector similarity alone misses structured, multi-hop relationships.
- **No quality visibility:** users cannot see extraction confidence or which entities are unreliable.

User pain: legal and compliance teams spend a large share of their time on document review, current AI tools are black boxes, and compliance work requires traceable analysis. **Market gap:** Microsoft GraphRAG is powerful but offline-oriented (slow, expensive to index); pure vector RAG loses cross-page relationships; few tools combine speed, auditability and accuracy.

3. Solution Overview

Three technologies fused behind one query router:

- **Vector embeddings (Qdrant):** semantic search over document chunks.
- **Knowledge graph (Neo4j):** structured relationship reasoning, multi-hop up to 3 hops.
- **LLM synthesis (Groq primary, Gemini free tier as fallback):** answer generation with inline citations.

Every answer ships with source citations (chunk + page), extraction confidence scores, and an interactive graph visualization of the entities and relationships behind the answer.

4. Target Users and Use Cases

Use case	What the user extracts
Legal / contracts	Parties, obligations, payment terms, dates
Financial	Regulatory filings, earnings reports, compliance documents
Research	Cross-paper relationships, author and citation graphs
Compliance	Policy adherence, regulatory alignment

5. MVP Scope

5.1 Included (with current status)

Area	Deliverable	Status
Backend	FastAPI skeleton, config, /health, Docker Compose (FastAPI + Neo4j + Qdrant)	DONE (Phase 1)
Backend	PDF parsing and chunking (PyMuPDF)	Phase 2
Backend	Entity and relationship extraction (LLM-based; spaCy optional)	Phase 2
Backend	Neo4j triplet storage + Qdrant vector indexing	Phase 3
Backend	Hybrid query router (parallel vector + graph)	Phase 4
Backend	LLM answer synthesis with citations	Phase 5
Frontend	Query form, answer display, vis.js graph, citation panel, metrics dashboard	Phase 6
Deployment	Docker Compose local; Railway (backend); Vercel (frontend)	Phase 7

5.2 Explicitly out of scope for MVP

- Multi-document reasoning
- Fine-tuned extraction models
- Temporal reasoning
- Cross-document link analysis
- Conversation history
- User authentication and multi-tenancy

6. Success Metrics (Targets)

Important: the figures below are design **targets**, not measured results. Any claim of achieved accuracy (for example to sponsors or customers) must come from the benchmark run on the 50-question test set in Phase 7.

Metric	Target	How measured
Factual accuracy	94% (floor 85%)	Human-verified Q&A; on 50 test questions across 10 documents
Citation coverage	98%	% of claims with a source link
Query latency	under 2 s	Average on 500-page documents
Extraction coverage	90%+	% of important entities found
Avg confidence score	0.85+	Mean confidence of extracted relationships

7. User Stories

- **Legal review:** As a compliance officer, I want to ask "What are all payment obligations in this contract?" so I can verify the AI did not hallucinate by seeing the exact clauses.
- **Document exploration:** As a researcher, I want to see which authors collaborate and on which topics so I can understand the knowledge structure visually.
- **Confidence-based review:** As a financial analyst, I want to know which extracted facts are low-confidence so I can manually review the risky ones.

8. Non-Functional Requirements

Area	Requirement
Security	API keys only in .env / platform secrets; uploaded PDFs processed in memory and not stored; only extracted data persisted to the databases
Performance	Query latency under 2 s on 500-page documents; ingestion about 8-10 s per 100 pages (acceptable for MVP)
Scalability	Single instance, about 100 concurrent users
Reliability	Graceful degradation when an LLM API or a database is unavailable (see SRS fallbacks)
Usability	Desktop and tablet; mobile optional
Cost	$0 running cost: free-tier APIs and self-hosted databases only

9. Dependencies and Cost (Updated)

Dependency	Role	Cost / limit
Groq API	Primary LLM (extraction + synthesis)	Free tier, rate limited
Google Gemini API	**Fallback LLM (replaces Claude API)**	Free tier via Google AI Studio, no card; confirm current model name and limits at implementation
Neo4j Community	Knowledge graph	Free, self-hosted (Docker)
Qdrant (self-hosted)	Vector search	Free, self-hosted; **no API key needed**. Qdrant Cloud free tier (1 GB RAM, 4 GB disk) is a backup option only
Railway / Vercel	Deploy backend / frontend	Free tiers

10. Roadmap

Phase	Goal	Days	Status
1	Foundational setup: FastAPI skeleton, env config, Docker Compose, health check	1-3	DONE
2	Ingestion: PDF parsing, chunking, entity/relationship extraction, confidence scoring	4-6	Next
3	Indexing: Neo4j triplet insertion, Qdrant vector indexing, metadata, query optimization	7-9	Planned
4	Query processing: vector search, NL-to-Cypher, graph search, result fusion	10-12	Planned
5	Synthesis: LLM answers, citation extraction, page lookup, confidence annotation	13-15	Planned
6	Frontend: React + Vite, query form, vis.js graph, citation panel, metrics	16-18	Planned
7	Integration, testing, benchmarks, deployment (Railway + Vercel)	19-21	Planned

11. Risks and Mitigations

Risk	Mitigation
Free-tier LLM rate limits slow ingestion or queries	Groq primary, Gemini fallback; chunk-level batching and retry with backoff; cache extraction results per chunk
Extraction quality (wrong or missing relationships)	Confidence scoring, confidence-threshold filtering, prompt iteration against the 50-question benchmark
Railway free tier (512 MB RAM) too small for FastAPI + Neo4j + Qdrant + embedding model	Validate early in Phase 7; fallback is Qdrant Cloud free tier and/or Neo4j AuraDB free tier, or a small VPS
Published accuracy numbers are unvalidated	Treat as targets until benchmark results exist; do not present as achieved

12. Change Log vs. Master Build Spec v1.0

- Fallback LLM changed from Anthropic Claude (paid) to Google Gemini free tier, to keep MVP cost at zero.
- Accuracy, citation and latency figures re-labelled as targets rather than results.
- Phase 1 marked complete; docker-compose uses a NEO4J_PASSWORD environment variable instead of a hardcoded password.
- Qdrant clarified: self-hosted needs no API key.